

Architectural Analysis of Open-Source Language Learning Systems: Duolingo-Style Frameworks, Morphological Engines, and Pedagogical Corpora

Computer-assisted language learning (CALL) systems increasingly rely on micro-learning paradigms, spaced repetition, and gamified reinforcement mechanics popularized by commercial applications such as Duolingo. Developing domain-specific platforms—specifically those targeting morphological awareness, prefix and suffix derivation, and phonics-based early literacy—requires a specialized architectural approach. Standard flashcard tools often fail to represent the compositional nature of morphemes, while monolithic language platforms can be overly rigid for dynamic lexical games.

Analyzing the open-source ecosystem reveals a functional division among available technologies: full-scale learning platforms modeled after Duolingo's macro curriculum; lightweight, client-side morphological engines designed for subword assembly; phonics-centered literacy applications that connect orthography with speech; and machine-readable lexical databases that supply the underlying derivational data.

Architectural Paradigms of Open-Source Duolingo Counterparts

Replicating Duolingo's core user experience involves coordinating modular skill trees, spaced repetition algorithms, interactive bite-sized challenges, and immediate gamified feedback loops. Several open-source projects provide complete application architectures for these mechanics, categorized by license, technology stack, and platform design.

Platform	Repository	License	Architectural Stack	Primary Educational Delivery Focus
LibreLingo	LibreLingoCommunity/LibreLingo	AGPL-3.0	Python YAML CLI, Svelte, PouchDB	Offline-first, community-authored language courses

UneeBee	zoonk/uneeb e	AGPL-3.0	Elixir, Phoenix LiveView, PostgreSQL	Interactive micro-lesson cards and progressive Q&A
LEAF	SudoKMaar/lea f	MIT	Next.js, TypeScript, Drizzle, Neon DB	Gamified language learning with hearts and leaderboards
Lingo	sanidhyy/duolin go-clone	MIT	Next.js, React, Zustand, Stripe, Drizzle	Multi-module challenge roadmaps and structured quizzes
Euolingo	ikyawthetpaing /euolingo	MIT	React Native, Expo, Firebase	Cross-platform mobile gamification and tracking

LibreLingo provides a decentralized, community-driven alternative to proprietary language platforms¹. The engine strictly isolates curriculum content from client presentation³. Course developers author curricula using structured YAML files and Markdown documentation, which are validated by the librelingo-yaml-loader Python package and compiled into optimized JSON bundles using librelingo-json-export³. The client interface uses Svelte and PouchDB to enable offline persistence and cross-device data synchronization³.

LibreLingo supports various exercise formats, including multiple-choice recognition, word-bank sentence assembly, and auditory dictation⁴. Dedicated modules, such as the Mandarin Chinese course (LibreLingo-ZH-from-EN), show how the platform can handle tonal systems, Pinyin annotations, and integrated text-to-speech (TTS) audio⁷. The compilation pipeline requires familiarity with Python command-line tools, but its file-based design makes it well-suited for version-controlled, collaborative curriculum development⁴.

UneeBee offers an alternative architecture based on the Elixir and Phoenix LiveView ecosystem⁹. Designed to lower technical barriers for interactive course authoring, it handles interface reactivity and state transitions on the server, avoiding heavy client-side JavaScript frameworks¹¹. Although the maintainer shifted focus to a subsequent platform (Zoonk) built on

Next.js, the archived UneeBee repository remains an adaptable reference for server-rendered interactive learning workflows¹¹. The system structures educational content into modular steps, combining reading elements with interactive review cards to support progressive disclosure⁹. A distinct group of permissively licensed Duolingo clones has emerged in the React and TypeScript ecosystems, notably SudoKMaar/leaf, sanidhyy/duolingo-clone (Lingo), and Davronov-Alimardon/duolingo-clone¹⁴. These systems implement web architectures using Next.js App Router, React Server Components, and Tailwind CSS¹⁴. User state, active sessions, and gamification loops—such as heart deductions, experience points, and streaks—are managed through lightweight Zustand stores¹⁴. Relational backends are typically powered by PostgreSQL (often via serverless providers like Neon) and accessed through Drizzle ORM¹⁴. The corresponding schemas map courses to units, units to lessons, and lessons to challenges with multiple options¹⁴. For developers building specialized morphological games, these MIT-licensed repositories provide functional UI primitives, reward loops, and administrative dashboards that can be adapted to evaluate morphemic elements rather than full sentences¹⁴.

Morphological Word-Building Games and Subword Assembly Engines

While general language platforms focus on whole-word recognition and syntactic translation, teaching morphology requires engines capable of handling combinatorial rules, subword affixation, and derivational families. A targeted set of open-source projects demonstrates how prefixes, roots, and suffixes can be gamified as interactive assembly mechanisms.

Word Root Workshop (joeseesun/word-root-workshop) is an open-source web application designed for morphological decomposition¹⁷. Built without external framework dependencies using native HTML5, modern CSS, and vanilla JavaScript, the system models vocabulary acquisition on modular construction blocks: Prefix + Root + Suffix¹⁷. The software organizes learning around 300 Classical Latin and Greek roots, which connect derivationally to thousands of English target words¹⁷.

The application stores data in `js/wordData.js`, defining roots alongside their origins, semantic definitions, example words, and component morphemes¹⁷. Words are broken down into their constituent parts; for example, *example* is mapped to the prefix *ex-* (out), the root *ample*, and an optional suffix, paired with an explanation of how the combined parts form the full definition¹⁷.

The interface provides three pedagogical loops: a structured learning mode with inline comprehension checks, a flashcard review mode with keyboard navigation, and an index tracking user progress via browser `LocalStorage`¹⁷. Because it has no external build dependencies, developers can fork the repository, modify the underlying JSON dictionary to support other languages, and deploy it statically to GitHub Pages¹⁷.

Alternative gameplay mechanics appear in smaller educational projects on GitHub. The browser-based card games by hebrews110 (prefixSuffix and prefixSuffixThird) demonstrate spatial, matching-based mechanics for affix education¹⁸. Players navigate a field of floating or

tilted cards containing isolated affixes (un-, re-, -able, -tion) and base words (do, play, construct)¹⁸. The game validates pairs based on legitimate morphological combinations rather than visual matching¹⁸.

A central challenge in these games is morphological overgeneration, where valid prefixes attach to multiple candidate bases. The game engine must either accept all linguistically valid pairings or structure selections so players do not strand necessary morphemes later in the exercise¹⁸.

Algorithmic word-formation mechanics also appear in projects like AcrossWords (jacobprall/CrossWords) and client-side anagram systems like Bennett McIntosh's Words²⁰. The AcrossWords engine pre-indexes database collections with substring arrays of valid prefixes

and suffixes, enabling lookups in $O(\log n)$ time to keep clue validation and board overlap responsive²⁰.

Similarly, solver systems such as WordBox-Solver-for-Plato use depth-first search (DFS) with prefix-tree (Trie) pruning to traverse letter matrices²². These indexing strategies provide a useful foundation for custom word-building games that need to evaluate dynamic player submissions against linguistic rules in real time.

Literacy, Phonics, and Speech Synthesis Systems

Teaching reading and oral pronunciation requires coordinated integration of graphemes, phonemes, and acoustic models. Unlike text-based flashcard applications, early literacy tools must directly connect spelling to speech sounds.

System	Repository / Organization	License	Technical Runtime	Pedagogical Focus
Feed The Monster	curiouslearning/FeedTheMonster	Open Source / DPG	Unity C#, Android Native	Phonics, letter-sound association, early reading
FeedTheMonsterJS	curiouslearning/FeedTheMonsterJS	MIT	TypeScript, HTML5 Canvas	Lightweight, browser-based phonics game loop
GCompris	KDE (invent.kde.org/education/gcompris)	GPL-3.0	C++, Qt, QML	Multilingual reading, orthography, and speech

Lute / LWT	Lute-Org/lute	Open Source	Python / PHP, SQLite	Reading comprehensio n via contextual text lookups
-------------------	---------------	-------------	-------------------------	--

Feed The Monster, designated as a Digital Public Good (DPG), is an open-source early literacy engine designed to teach reading mechanics through gamified reinforcement²³. Developed by Curious Learning, the original system was implemented in Unity C# and later adapted for the web using TypeScript and HTML5 Canvas (FeedTheMonsterJS)²³.

The game loop focuses on phonological awareness: an animated monster character prompts the user for specific letter sounds or syllables, and the player identifies the matching tiles from a moving collection²³. Successfully dragging the correct phoneme into the monster's mouth triggers visual and auditory rewards that reinforce the grapheme-phoneme link²³.

The engine uses a modular asset structure, managed through separate language repositories (ftm-languagepacks)²⁴. This separation allows developers to update localized audio recordings, script assets, and progression levels for different languages without altering the underlying code²⁴. The TypeScript and Canvas port provides an accessible codebase for web deployment, making it an effective model for games that link subword combinations to spoken audio²⁴.

At a broader institutional level, the KDE Community maintains GCompris, a comprehensive multiplatform educational suite containing over one hundred activities²⁶. Built with C++ and Qt Quick/QML, GCompris includes activities for reading practice, word construction, letter tracing, and vocabulary acquisition²⁶.

The project maintains an open repository of high-quality, localized voice recordings and JSON-based word catalogs (gcompris-data), covering dozens of languages²⁷. The modular design of its QML components offers useful patterns for building drag-and-drop word builders backed by native audio feedback²⁶.

Developers building web-based language tools can also implement modern browser speech APIs rather than relying entirely on pre-recorded audio files. The Web Speech API provides built-in text-to-speech (window.speechSynthesis) and speech recognition (webkitSpeechRecognition) across major browsers at zero infrastructure cost.

For server-side or pre-compiled audio generation, open-source libraries such as edge-tts (which interfaces with high-quality neural voices) and local Coqui TTS models provide programmatic pipelines for generating clear pronunciation samples for newly assembled words².

Curated Linguistic Datasets and Morphological Corpora

An educational game that teaches prefix and suffix combinations requires a structured

dictionary defining valid morphemic pairings. Building these lists manually is labor-intensive, but several academic and computational databases offer accessible, machine-readable derivational data.

Dataset / Resource	Source Repository / Host	Data Format	Linguistic Content and Coverage
MorphoLex-en	hugomailhot/MorphoLex-en	XLSX, CSV	68,624 English words with derivational and morphemic features
MorphoLex-fr	hugomailhot/morpholex-fr	CSV	38,840 French words with Prefix-Root-Suffix signatures
English Roots List	WithEnglishWeCan/generated-english-roots-list	Markdown, CSV	Classical Latin/Greek roots, affixes, meanings, and examples
Etymology-DB	droher/etymology-db	Parquet, CSV	Graph-based relational morpheme links extracted from Wiktionary
Wiktextextract	kambanaria/wiktextextract	JSON Lines	Over 8 million machine-readable lexical entries across 300+ languages

The MorphoLex project provides detailed, research-grade derivational databases for English and French²⁹. The English dataset (MorphoLex-en) covers 68,624 words drawn from the English Lexicon Project, while the French version (MorphoLex-FR) includes 38,840 words³⁰.

These databases provide structured fields that translate directly into game mechanics: The `prs` attribute categorizes words using an explicit Prefix-Root-Suffix signature (e.g., `0_1_1` denotes an

un-prefixed word with a single root and suffix), while `canon_segm` provides the morphological segmentation of each word into its constituent parts³⁰.

The database also records psycholinguistic frequency metrics for prefixes (`PREF_Freq_HAL`), roots (`ROOT_Freq_HAL`), and suffixes (`SUFF_Freq_HAL`), along with derivational family sizes³⁰.

These frequency statistics allow developers to balance game difficulty programmatically, introducing common affixes like `-er` or `un-` before less frequent, specialized morphemes³¹.

The generated-english-roots-list project provides an educational dataset mapping Latin and Greek roots to prefixes, suffixes, concise definitions, and sample vocabulary words³⁴. Because entries are organized around clear derivational links (e.g., mapping `ab-` to *abbreviate*, *abnormal*, and *abstract*), this dataset is well suited for generating dynamic matching challenges and multiple-choice questions³⁴.

For multilingual applications, `etymology-db` provides a large-scale relational extraction of Wiktionary's etymological entries, formatted in Parquet and CSV³⁵. The dataset models morphology as a directed graph using explicit relation tags³⁵.

The `has_prefix` and `has_suffix` attributes map derived words to their respective affixes, while `has_prefix_with_root` and `has_suffix_with_root` identify the companion stem³⁵. The dataset also includes `has_confix` for bound morphemes and `compound_of` for compound words³⁵.

These structured relationships allow developers to query valid combinations dynamically or generate word-family challenges across multiple target languages using a local SQLite or PostgreSQL database³⁵.

Comparative Systems Evaluation

Selecting the appropriate open-source base requires weighing architectural complexity, licensing restrictions, and pedagogical scope.

Candidate Project	License	Tech Stack & Setup Overhead	Gameplay & Pedagogical Scope	Integration Feasibility
Word Root Workshop [cite: 17]	Permissive / Open	Pure HTML/CSS/Vanilla JS; zero build dependencies; offline storage	Subword Prefix+Root+Suffix assembly, flashcards, instant quizzes	High: Can be reused directly, styled easily, and customized via JSON
FeedTheMonsterJS [cite: 24]	MIT	TypeScript, HTML5 Canvas; medium setup; localized audio	Phonics recognition, speech reinforcement, character	Moderate: Game loop is tied to Canvas; assets easily adaptable

		assets	progression	
LEAF / Lingo [cite: 14, 16]	MIT	Next.js, React, Tailwind, Drizzle, Neon DB; full-stack configuration	Complete Duolingo experience: streaks, hearts, leagues, quizzes	Moderate-High: Requires backend setup; UI patterns readily adaptable
LibreLingo [cite: 3, 4]	AGPL-3.0	Python CLI, Svelte, PouchDB; requires YAML parsing and asset setup	Multi-language course progression, spaced repetition, TTS audio	Moderate: Strong CLI toolchain, but AGPL-3.0 requires open derivatives
UneeBee [cite: 11]	AGPL-3.0	Elixir, Phoenix LiveView, PostgreSQL; requires Erlang environment	Card-based interactive steps, progressive Q&A disclosure	Low-Moderate: Archived codebase; requires an Elixir backend
MorphoLex Databases [cite: 29, 30]	Open Research / CC	Static CSV / XLSX tables; zero computational runtime	Provides lexical verification, morpheme boundaries, and frequency data	High: Integrates directly into database seeds or client JSON caches

These technical trade-offs suggest that monolithic language platforms like LibreLingo or full-stack Duolingo clones offer comprehensive overall architectures, but their core data models are centered on sentences and translation pairs⁴. Adapting them to teach morphological derivation requires modifying their challenge generation logic and database schemas¹⁴.

Conversely, single-purpose projects like Word Root Workshop directly implement prefix-root-suffix mechanics and can be deployed statically, but they lack advanced gamification features like global leaderboards or automated speech synthesis¹⁷.

Permissively licensed codebases (MIT) such as Word Root Workshop, FeedTheMonsterJS, and the Next.js clones (LEAF, Lingo) offer the greatest flexibility for building custom systems

without the copyleft constraints of AGPL-3.0 platforms¹⁴.

Technical Recommendations for an Affix-Combining Game

Building an educational word-assembly game that teaches morphological structure, reading, and pronunciation is best achieved by combining specialized, modular components rather than modifying a single large platform. An effective architecture links an interactive presentation layer, a linguistic validation engine, and an automated speech pipeline.

Architectural Component Layers



The presentation layer manages user interaction and game state. For developers prioritizing simplicity and low maintenance, adopting the vanilla JavaScript architecture from Word Root Workshop allows the interface to run statically on any web host¹⁷.

If broader user management, leaderboards, and persistent cross-device progress are required, wrapping these mechanics in an MIT-licensed Next.js framework like LEAF or Lingo provides a tested frontend foundation¹⁴.

The user interface should center on three designated morpheme slots—Prefix, Root, and

Suffix—accompanied by a dynamic tray of draggable or clickable tiles¹⁷. As learners connect affixes to base stems, the interface can dynamically display the evolving meaning, illustrating how prefixes alter direction or negation and suffixes determine grammatical function¹⁷.

The linguistic validation layer evaluates user input in real time. Developers can generate a consolidated JSON dictionary by combining the derivational metrics of MorphoLex-en with the definitions and examples from generated-english-roots-list²⁹.

The game engine can organize these combinations into a prefix tree (Trie), enabling the system to validate subword sequences instantly as tiles are placed²². When an affix causes an orthographic change—such as deleting the final silent e before -able or changing y to i—the validation logic can apply replacement rules, teaching learners standard spelling conventions alongside morpheme recognition.

The audio and speech layer reinforces reading and pronunciation by giving players immediate acoustic feedback for completed words. Basic spoken playback can be implemented at no cost using the browser-native Web Speech API (`window.speechSynthesis`), specifying the target locale.

For higher acoustic quality or less common languages, developers can follow the pattern used by Feed The Monster and GCompris, assembling pre-recorded phoneme segments or using open-source tools like edge-tts to batch-produce neural audio assets².

Pronunciation practice can similarly use the Web Speech Recognition API (`webkitSpeechRecognition`), prompting learners to read assembled words aloud and comparing the transcript against the target lemma.

Recommended Implementation Sequence

A practical integration workflow proceeds through four phases:

First, extract the subword assembly logic and data schema from Word Root Workshop (joeseesun.com/word-root-workshop) to establish the fundamental Prefix + Root + Suffix interaction model¹⁷.

Second, convert the derivational categories and morpheme signatures from MorphoLex-en and generated-english-roots-list into a normalized JSON dictionary, organizing entries by semantic definition, constituent morphemes, and frequency-based difficulty levels²⁹.

Third, integrate these mechanics into an MIT-licensed web framework, such as the LEAF or Lingo templates, to incorporate gamification elements like experience points, streak counters, and health systems¹⁴.

Fourth, implement speech feedback using the Web Speech API for real-time text-to-speech, supplemented by phonics-oriented audio cues inspired by FeedTheMonsterJS to connect written morphemes with spoken language²³.

This modular approach avoids the overhead and licensing challenges of large monolithic frameworks, providing a reliable path to building an educational word game focused on morphology, reading, and pronunciation.

Works cited

1. LibreLingo - Open source language learning platform,

- <https://szabgab.com/librelingo>
2. Vuizur/awesome-language-learning - GitHub, <https://github.com/Vuizur/awesome-language-learning>
 3. kantord/LibreLingo: a community-owned language-learning ... - GitHub, <https://github.com/kantord/LibreLingo>
 4. LibreLingo a community-owned language-learning platform - GitHub, <https://github.com/LibreLingoCommunity/LibreLingo>
 5. LibreLingo/Dockerfile at main - GitHub, <https://github.com/LibreLingoCommunity/LibreLingo/blob/main/Dockerfile>
 6. When redoing a "word picking" task, words go missing #25 - GitHub, <https://github.com/LibreLingo/LibreLingo-ES-from-EN/issues/25>
 7. GitHub - aman-lj/LibreLingo-ZH-fron-EN, <https://github.com/aman-lj/LibreLingo-ZH-fron-EN>
 8. GitHub - LibreLingoCommunity/course-template, <https://github.com/LibreLingoCommunity/course-template>
 9. Open-source tool for building courses like Duolingo - Reddit, https://www.reddit.com/r/SideProject/comments/17y2bfb/opensource_tool_for_building_courses_like_duolingo/
 10. Open-source tool for Duolingo-like courses - Indie Hackers, <https://www.indiehackers.com/post/open-source-tool-for-duolingo-like-courses-fcb872376e>
 11. GitHub - zoonk/uneebie: Platform for creating interactive courses., <https://github.com/zoonk/uneebie>
 12. zoonk repositories - GitHub, <https://github.com/orgs/zoonk/repositories>
 13. Zoonk - GitHub, <https://github.com/zoonk>
 14. LEAF: Full-stack Duolingo Clone Web Application - GitHub, <https://github.com/SudoKMaar/leaf>
 15. Davronov-Alimardon/duolingo-clone - GitHub, <https://github.com/Davronov-Alimardon/duolingo-clone>
 16. sanidhy/duolingo-clone: Lingo - Interactive platform for language, <https://github.com/sanidhy/duolingo-clone>
 17. joeseesun/word-root-workshop: 词根词缀记忆工坊 - GitHub, <https://github.com/joeseesun/word-root-workshop>
 18. Match Game, <https://hebrews110.github.io/prefixSuffixThird/>
 19. Match Game - GitHub Pages, <https://hebrews110.github.io/prefixSuffix/>
 20. jacobprall/CrossWords: MERN Stack Game - GitHub, <https://github.com/jacobprall/CrossWords>
 21. benn-mctosh/words: game for grandma - GitHub, <https://github.com/benn-mctosh/words>
 22. UmerTariq1/WordBox-Solver-for-Plato: Given a boggle grid ... - GitHub, <https://github.com/UmerTariq1/WordBox-Solver-for-Plato>
 23. Curious Learning - Digital Public Goods Backlog, <https://backlog.digitalpublicgoods.net/curiouslearning/>
 24. Curious Learning - GitHub, <https://github.com/curiouslearning>
 25. Activity · curiouslearning/FeedTheMonster - GitHub,

- <https://github.com/curiouslearning/FeedTheMonster/activity>
26. Software:GCompris - HandWiki, <https://handwiki.org/wiki/Software:GCompris>
 27. Developer's corner · Wiki · Education / GCompris · GitLab - KDE Invent, <https://invent.kde.org/education/gcompris/-/wikis/Developers-corner>
 28. Education / GCompris Data · GitLab - KDE Invent, <https://invent.kde.org/education/gcompris-data>
 29. hugomailhot/MorphoLex-en: Lexical database for - GitHub, <https://github.com/hugomailhot/MorphoLex-en>
 30. GitHub - hugomailhot/morpholex-fr: Lexical database for, <https://github.com/hugomailhot/morpholex-fr>
 31. MorphoLex: A derivational morphological database ... - ResearchGate, https://www.researchgate.net/publication/320987123_MorphoLex_A_derivational_morphological_database_for_70000_English_words
 32. 0_1_1.csv - hugomailhot/morpholex-fr - GitHub, https://github.com/hugomailhot/morpholex-fr/blob/master/csv/0_1_1.csv
 33. A derivational morphological database for 70000 English words, <https://pubmed.ncbi.nlm.nih.gov/29124719/>
 34. WithEnglishWeCan/generated-english-roots-list - GitHub, <https://github.com/WithEnglishWeCan/generated-english-roots-list>
 35. droher/etymology-db - GitHub, <https://github.com/droher/etymology-db>